

# LangChain: Modeller, Prompt Şablonları ve Çıktı Ayırıştırıcılar

## Yapay Zeka Uygulama Geliştirme Çalışma Raporu

Büyük Dil Modelleri (LLM) ile uygulama geliştirirken, tekrar kullanılabilirlik, prompt yapısının verimli yönetimi ve model çıktılarının yazılım sistemlerinde doğrudan işlenebilecek formatlara dönüştürülmesi en kritik süreçlerdir. LangChain, bu süreci kolaylaştırmak amacıyla üç temel soyutlama sunar: **Modeller**, **Promptlar (Şablonlar)** ve **Ayrıştırıcılar (Parsers)**. Bu rapor, DeepLearning.AI bünyesinde sunulan 'LangChain for LLM Application Development' eğitiminin ilk modülünden derlenen pratik bilgiler, kod örnekleri ve günlük hayattan senaryolar ile bu üç bileşenin nasıl entegre edildiğini açıklamaktadır.

- 1. Modeller (Models):** Uygulamanın temelini oluşturan, metin üreten veya sohbet arayüzü sağlayan yapay zeka dil modelleridir (Örn: gpt-3.5-turbo).
- 2. Prompt Şablonları (Prompts):** Modele gönderilecek girdilerin dinamik, parametrik ve tekrar kullanılabilir şekilde yapılandırılmasını sağlayan kalıplardır.
- 3. Çıktı Ayırıştırıcılar (Parsers):** LLM'den dönen serbest metin (string) halindeki yanıtları, JSON gibi yapılandırılmış verilere (Python sözlükleri) dönüştüren araçlardır.

## 1. Temel Kurulum ve Yardımcı Fonksiyon

Geliştirmeye başlamak için öncelikle gerekli kütüphanelerin içeri aktarılması ve API anahtarının tanımlanması gerekir. LangChain olmadan, doğrudan OpenAI API'sini kullanan temel bir yardımcı fonksiyon şu şekilde tanımlanır:

```
import os
import openai

# API Anahtarının Yüklenmesi
openai.api_key = os.getenv('OPENAI_API_KEY')

# Geleneksel Yardımcı Fonksiyon (ChatCompletion)
def get_completion(prompt, model="gpt-3.5-turbo"):
    messages = [{"role": "user", "content": prompt}]
    response = openai.ChatCompletion.create(
        model=model,
        messages=messages,
        temperature=0, # Çıktının rastgeleliğini sıfırlamak için 0 olarak ayarlanır
    )
    return response.choices[0].message["content"]
```

## 2. Prompt Şablonları ve Günlük Hayattan Çeviri Senaryosu

**Senaryo:** Küresel ölçekte hizmet veren bir e-ticaret siteniz olduğunu ve dünyanın dört bir yanından farklı dillerde veya üsluplarda (Örn: Korsan İngilizcesi) müşteri e-postaları aldığınızı düşünün. Müşteri hizmetleri temsilcilerinizin bu mesajları anlayabilmesi için gelen metinlerin sakın, saygılı ve standart bir Amerikan İngilizcesine dönüştürülmesi gerekmektedir.

### Gelen Korsan Mesajı:

*"I'd be fuming that me blender lid flew off and splattered my kitchen walls with smoothie. And to make matters worse, the warranty don't cover the cost of cleaning up me kitchen. I need your help right now, matey!"*

Python'ın standart **f-string** yapısını kullanarak bu problemi çözmek istediğimizde, promptu manuel olarak birleştirmemiz gerekir. Ancak bu yöntem, yüzlerce farklı dil ve stil kombinasyonu için içine girdiğinde karmaşıklaşır ve yönetilemez hale gelir:

```
# Hedef Stil ve Girdi Mesajı
style = "American English in a calm and respectful tone"
customer_email = "I'd be fuming that me blender lid flew off..."

# Klasik f-string ile Prompt Oluşturma
prompt = f"""Translate the text that is delimited by triple backticks \
into a style that is {style}.
Text: ```{customer_email}```
"""
response = get_completion(prompt)
```

LangChain, bu dinamik prompt oluşturma sürecini **ChatPromptTemplate** soyutlamasıyla optimize eder. Şablon bir kez tanımlanır, girdi değişkenleri otomatik olarak ayrıştırılır ve güvenli bir şekilde modellerle paylaşılır. Ayrıca sıfır sıcaklık (**temperature = 0.0**) ayarı ile modelin rastgele tahminler yapması önlenir:

```

from langchain_openai import ChatOpenAI
from langchain.prompts import ChatPromptTemplate

# 1. Sıcaklığı 0.0 olan LangChain Model Abstraksiyonunun kurulması
chat = ChatOpenAI(temperature=0.0, model="gpt-3.5-turbo")

# 2. Yeniden Kullanılabilir Şablonun Tanımlanması
template_string = """Translate the text delimited by triple backticks \
into a style that is {style}.
Text: ```{text}```
"""

prompt_template = ChatPromptTemplate.from_template(template_string)

# Şablon parametrelerinin otomatik algılanması
# Çıktı: ['style', 'text']
input_vars = prompt_template.input_variables

# 3. Şablonun belirli değerlerle doldurulması
customer_messages = prompt_template.format_messages(
    style="American English in a calm and respectful tone",
    text=customer_email)

# 4. Modele gönderilmesi ve yanıtın alınması
customer_response = chat.invoke(customer_messages)
print(customer_response.content)

```

### Karşılaştırma Analizi: Geleneksel f-string vs. LangChain Prompt Şablonu

| Özellik                  | Geleneksel Python f-string                                  | LangChain PromptTemplate                                                         |
|--------------------------|-------------------------------------------------------------|----------------------------------------------------------------------------------|
| Tekrar Kullanılabilirlik | Zor; her prompt için manuel f-string birleştirme gerekir.   | Kolay; şablon bir kez yazılır, farklı girdilerle dinamik çalıştırılır.           |
| Parametre Algılama       | Bulunmaz; parametrelerin kontrolü geliştiriciye aittir.     | Otomatik; input_variables özelliğiyle değişkenleri kendisi çıkarır.              |
| Entegrasyon Gücü         | Sadece metindir; zincirler veya harici API'lere bağlanamaz. | LangChain ekosistemindeki zincirlere (Chains) ve araçlara doğrudan entegre olur. |
| Hazır Yapılar            | Yoktur; her şeyi sıfırdan yazmanız gerekir.                 | Özetleme, Soru-Cevap, SQL bağlantısı vb. için hazır şablonlar sunar.             |

### 3. Çıktı Ayırıştırıcılar (Output Parsers) ve JSON Veri Çıkarımı

**Senaryo:** Müşterilerin ürünleriniz hakkında yazdığı serbest metin yorumlardan (Review) belirli bilgileri yapılandırılmış (Structured) şekilde çıkarmak istiyorsunuz. Örneğin bir yaprak üfleme makinesi (Leaf Blower) yorumundan şu 3 alanı çekmek istiyorsunuz: 1. Ürün hediye mi alındı? (True/False), 2. Kaç günde teslim edildi? (Sayı), 3. Fiyat/performans algısı (Metin).

#### Gelen Ürün Yorumu:

"This leaf blower is pretty amazing. It has four settings: candle blower, gentle breeze, windy city, and tornado. It arrived in two days, just in time for my wife's anniversary present. I think my wife liked it so much she was speechless..."

**Temel Problem:** Modele çıktıları JSON formatında vermesini söylesek bile, LLM'den dönen yanıt Python için **string (metin dizesi)** tipindedir. Bu string'i doğrudan bir Python sözlüğü (dictionary) gibi anahtarlarla (Örn: `response['gift']`) sorgulamaya çalışmak **hata (Error)** fırlatır. İşte bu noktada LangChain Çıktı Ayırıştırıcıları devreye girer.

```
from langchain.output_parsers import ResponseSchema
from langchain.output_parsers import StructuredOutputParser

# 1. Çıkarılacak alanların şemalarının (ResponseSchema) tanımlanması
gift_schema = ResponseSchema(
    name="gift",
    description="Was the item purchased as a gift for someone else? Answer True or yes, False if not or unknown."
)
delivery_days_schema = ResponseSchema(
    name="delivery_days",
    description="How many days did it take to deliver? Output only an integer."
)
price_value_schema = ResponseSchema(
    name="price_value",
    description="Extract any sentences about the value or price."
)

response_schemas = [gift_schema, delivery_days_schema, price_value_schema]

# 2. Şemalara dayalı Yapılandırılmış Çıktı Ayırıştırıcının kurulması
output_parser = StructuredOutputParser.from_response_schemas(response_schemas)

# 3. Model için özel biçimlendirme talimatlarının otomatik üretilmesi
format_instructions = output_parser.get_format_instructions()

# 4. Biçimlendirme talimatlarının prompt şablonuna gömülmesi
review_template = """\
For the following text, extract the following information:
gift: Was the item purchased as a gift for someone else? \
Answer True or yes, False if not or unknown.
delivery_days: How many days did it take to deliver? \
Output only an integer.
price_value: Extract any sentences about the value or price.

text: {text}

{format_instructions}
"""

prompt_template = ChatPromptTemplate.from_template(review_template)

# 5. Promptu oluşturup modele gönderme
messages = prompt_template.format_messages(
    text=customer_review,
    format_instructions=format_instructions
)
response = chat.invoke(messages)

# 6. STRING YANITIN GERÇEK BİR PYTHON SÖZLÜĞÜNE DÖNÜŞTÜRÜLMESİ
output_dict = output_parser.parse(response.content)

# Artık verilere güvenli bir şekilde erişebiliriz:
is_gift = output_dict.get("gift") # Çıktı: True (Boolean)
days = output_dict.get("delivery_days") # Çıktı: 2 (Integer)
```

## Çıktı Ayırıştırıcı (Output Parser) Akış Şeması

| Aşama                  | Adım Açıklaması                                                                                   | Kullanılan Yapı / Sınıf               |
|------------------------|---------------------------------------------------------------------------------------------------|---------------------------------------|
| 1. Şema Tanımlama      | Çıkarılacak alanların ismi ve detaylı açıklamaları şema nesneleri olarak tanımlanır.              | ResponseSchema                        |
| 2. Parser Kurulumu     | Tanımlanan şemalar birleştirilerek yapılandırılmış bir parser oluşturulur.                        | StructuredOutputParser                |
| 3. Talimat Üretimi     | Modelin çıktıyı tam olarak hangi JSON formatında vermesi gerektiğini anlatan talimatlar üretilir. | parser.get_format_instructions()      |
| 4. Dinamik Prompt      | Üretilen talimatlar ve girdi metni prompt şablonuna yerleştirilip LLM'e iletilir.                 | ChatPromptTemplate & chat.invoke()    |
| 5. Ayırıştırma (Parse) | LLM'den dönen string tipi metin, hatasız bir şekilde Python sözlük (dict) yapısına dönüştürülür.  | output_parser.parse(response.content) |

## 4. Sonuç ve Özet

LangChain'in sağladığı Modeller, Prompt Şablonları ve Çıktı Ayırıştırıcıları, LLM tabanlı uygulamalarda geliştirme sürecini standartlaştırarak büyük bir verimlilik sağlar. Dinamik prompt şablonları sayesinde uzun ve karmaşık talimatlar kolayca yönetilip yeniden kullanılabilirken; Çıktı Ayırıştırıcılar sayesinde de modellerin ürettiği metinler, geleneksel yazılım sistemlerinin (veritabanları, API'ler, mikroservisler vb.) doğrudan işleyebileceği yapısal nesnelere güvenli bir şekilde dönüştürülür. Bu üçlü yapı, LLM'leri bağımsız sohbet araçları olmaktan çıkarıp, karmaşık kurumsal uygulamaların güvenilir birer bileşeni haline getirmenin anahtarıdır.

### Çalışma Raporu Künyesi:

- **Kurs:** LangChain for LLM Application Development (DeepLearning.AI)
- **Eğitmenler:** Harrison Chase (LangChain Kurucusu), Andrew Ng
- **Konu:** Models, Prompts, and Parsers
- **Rapor Hazırlayan:** Gemini Notebook Geliştirici Asistanı